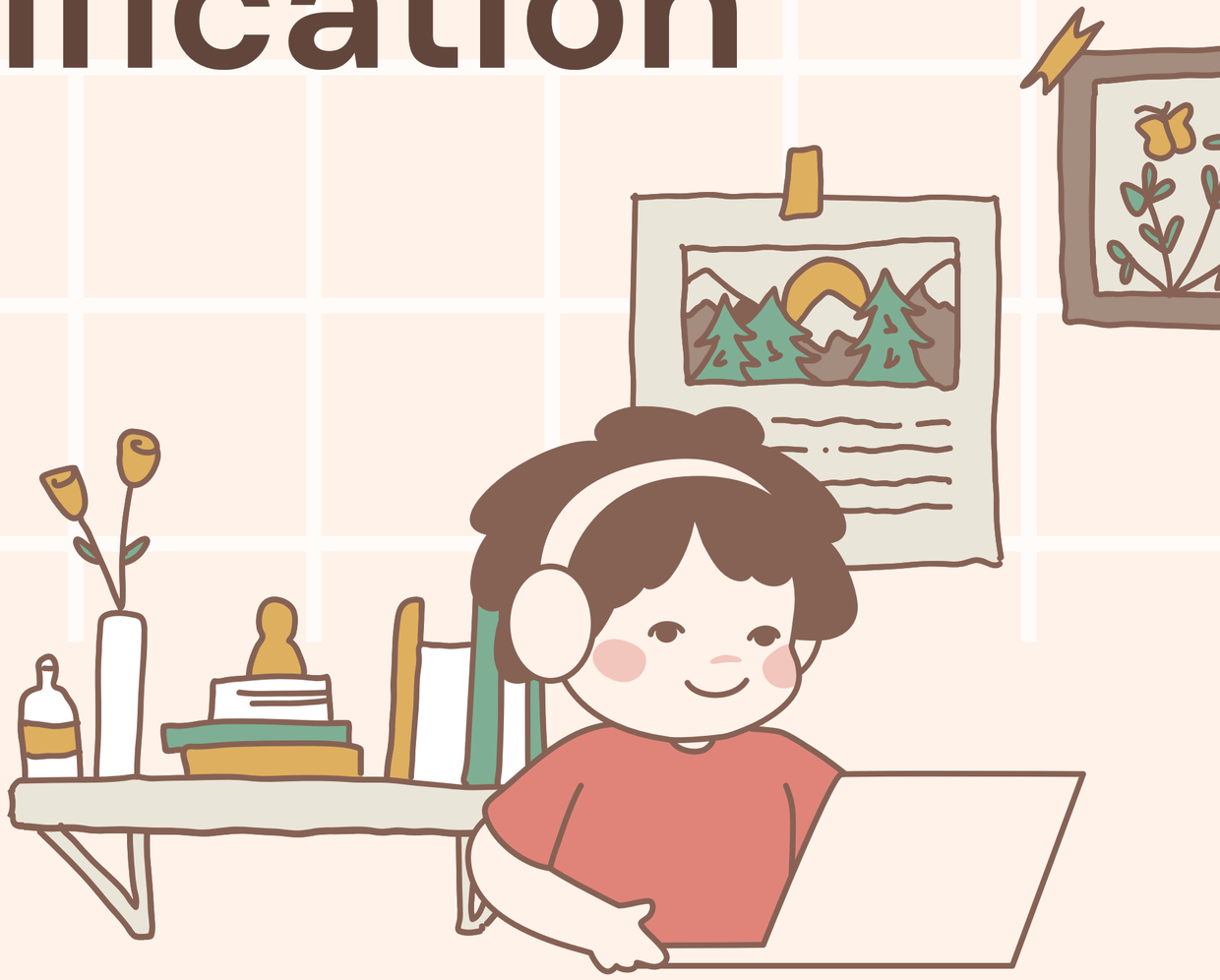
Log Loss: Cross-Entropy Loss in Classification

Understanding Probabilistic Evaluation for Classifiers



Tiffany Owen

2024 January

Table of Contents

What is Log Loss? 1

Why Use Log Loss? 2

Intuition Behind Log Loss 3

The Formula Explained 4

Evaluating Probabilities 5

Prediction Outcomes 6

Log Loss

What is Log Loss?

Log Loss (Logarithmic Loss), also called Cross-Entropy Loss, is a metric that measures how well a classification model predicts probabilities. Unlike accuracy, it penalizes confident wrong predictions heavily.

Why Use Log Loss in Classification?

Accuracy only checks if a prediction is right or wrong. Log Loss evaluates the quality of predicted probabilities — rewarding confident correct predictions and heavily penalizing confident mistakes. Lower Log Loss means a better-calibrated model.



Log Loss: The Intuition

Log Loss penalizes models based on how confident they are in their predictions. The more confident a wrong prediction, the greater the penalty — rewarding honest, well-calibrated probabilities.



High probability → correct class



Low probability → correct class



Confidently wrong → max penalty

Binary Log Loss Formula Breakdown

The True Label (y)

y is the true class label for each example. In binary classification, y equals 1 if the example belongs to the positive class, and 0 if it belongs to the negative class. The formula uses y and (1 - y) as switches: when y = 1, only the first term log(p) is active; when y = 0, only the second term log(1 - p) is active. This elegantly handles both classes in a single expression.

Predicted Probability (p)

p is the model's predicted probability that an example belongs to the positive class ($0 < p < 1$). The formula penalizes predictions using the logarithm: -log(p) is small when p is close to 1 (confident and correct), and very large when p is close to 0 (confident and wrong). The term log(1 - p) mirrors this for the negative class, penalizing high p when the true label is 0.

Averaging Across Examples

The full formula is: Loss = $-(1/N) \times \Sigma [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$. The sum runs over all N training examples, accumulating the per-sample loss. Dividing by N gives the average loss per example, making the metric comparable across datasets of different sizes. The negative sign ensures the result is positive, since log values between 0 and 1 are always negative.

Log Loss vs. Accuracy: How They Evaluate Models

Log Loss

Evaluates predicted probabilities, not just final class labels.
Penalizes confident wrong predictions heavily.
Sensitive to how certain the model is — rewards well-calibrated confidence.

VS

Accuracy

Evaluates only the final predicted class (0 or 1).
Treats all correct predictions equally, regardless of confidence.
Can be misleading when class distribution is imbalanced.

Log Loss: Model Confidence

High-Confidence Correct Predictions

When a model assigns a high probability (e.g., 0.95) to the correct class, the log loss contribution is very small — close to zero. The model is rewarded for being both correct and confident. For example: if y=1 and p=0.95, loss = -log(0.95) ≈ 0.051. This near-zero penalty reflects ideal model behavior.

Low Probability Correct Predictions & Confidently Wrong

When the model assigns a low probability to the correct class (e.g., p=0.1), the penalty is large: -log(0.1) ≈ 2.30. Worst case: a confidently wrong prediction (e.g., y=1 but p≈0.01) yields -log(0.01) ≈ 4.60 — a severe penalty. Log Loss heavily punishes overconfident mistakes, making it a sensitive and honest measure of model quality.



Log Loss: Step-by-Step Example

Sample Predictions

Suppose we have 4 binary samples. True labels: y = [1, 1, 0, 0]. Model predicted probabilities: p = [0.9, 0.2, 0.1, 0.8].
Sample 1: y=1, p=0.90
Sample 2: y=1, p=0.20
Sample 3: y=0, p=0.10
Sample 4: y=0, p=0.80
For each sample we apply the Log Loss formula:
$-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$

Individual Loss Values

Sample 1 (y=1, p=0.90):
$-\log(0.90) \approx 0.105$
Model is confident & correct → low loss.
Sample 2 (y=1, p=0.20):
$-\log(0.20) \approx 1.609$
Model is uncertain & wrong → high loss.
Sample 3 (y=0, p=0.10):
$-\log(1-0.10) = -\log(0.90) \approx 0.105$
Model is confident & correct → low loss.
Sample 4 (y=0, p=0.80):
$-\log(1-0.80) = -\log(0.20) \approx 1.609$
Model is confidently wrong → very high loss.

Average Log Loss

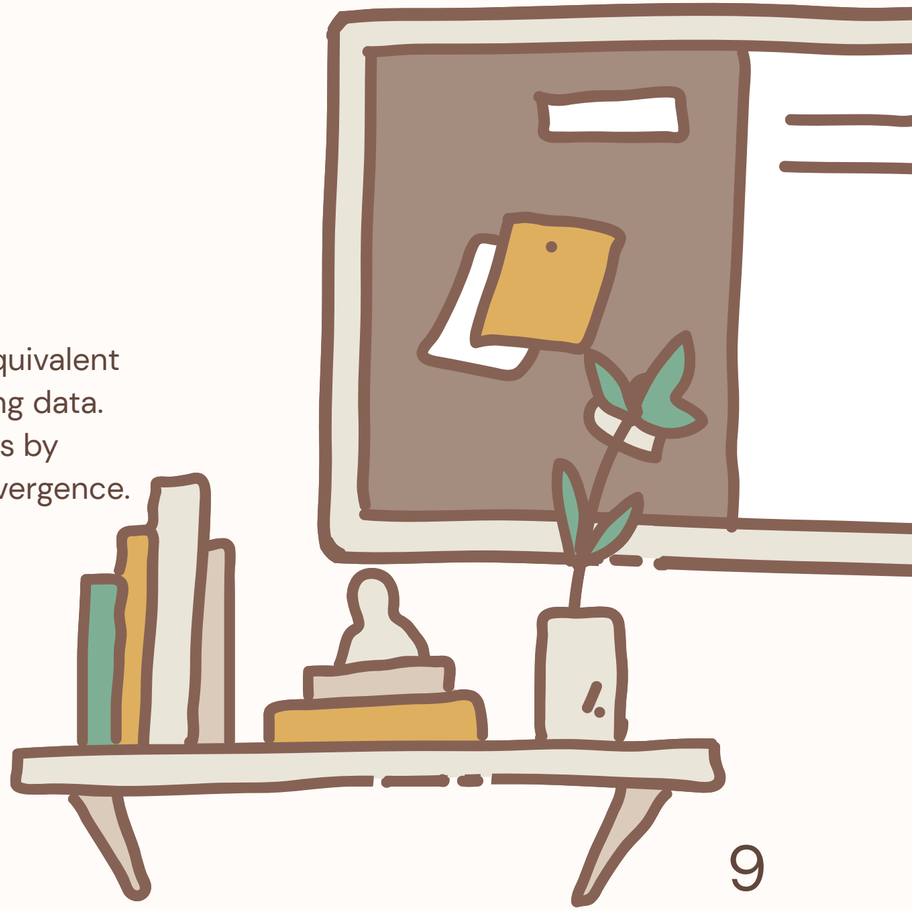
Sum all individual losses:
0.105 + 1.609 + 0.105 + 1.609 = 3.428
Divide by number of samples (n=4):
Log Loss = 3.428 / 4 ≈ 0.857
Interpretation: A Log Loss of 0 is perfect. Values closer to 0 indicate better probability estimates. Here, two confidently wrong predictions (Samples 2 & 4) dominate and push the average loss high — demonstrating how Log Loss heavily penalises confident mistakes.

Log Loss & Logistic Regression

Logistic Regression uses the sigmoid function to output probabilities, and Log Loss serves as its optimization objective during training.

The sigmoid function maps any real number to a value between 0 and 1, producing a valid probability. Training minimizes Log Loss via Maximum Likelihood Estimation (MLE), making predictions as probable as possible.

Minimizing Log Loss is mathematically equivalent to maximizing the likelihood of the training data. Gradient descent updates model weights by following the Log Loss gradient until convergence.



Advantages & Limitations of Log Loss

Advantages

Sensitive to predicted probabilities, not just class labels.
Rewards well-calibrated confidence in correct predictions.
Penalizes overconfident wrong predictions heavily, encouraging caution.
Aligns naturally with maximum likelihood estimation in logistic regression.

VS

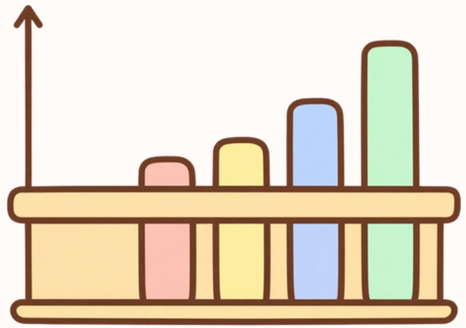
Limitations

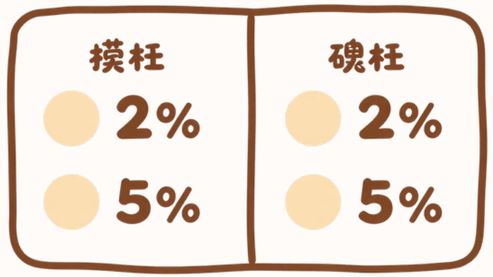
Inflicts very strong penalties when the model is confidently wrong.
Requires reliable, well-calibrated probability outputs to be meaningful.
Harder to interpret intuitively compared to accuracy.
Sensitive to noisy labels and outliers in the dataset.

Python Implementation of Log Loss

Python makes it easy to compute Log Loss, evaluate predicted probabilities, and compare it with Accuracy using sklearn and custom code.







Compute Log Loss with sklearn

Evaluate Predicted Probabilities

Compare Log Loss vs. Accuracy

Conclusions

Use Log Loss to Assess Probability Quality:
Not just what the model predicts, but how confident and correct it is





Pair Log Loss with Accuracy:
Interpret confidence carefully — a model can be accurate yet poorly calibrated

The End

Key Takeaway: Good models are not only correct — they are appropriately confident.

